

Practice Programs — Problem Solving, C Basics & Operators

This set contains **21 practice programs** mapped to the syllabus units below. Each entry gives the **problem statement** first, followed by the **algorithm / solution program** and a line-by-line **explanation**.

Unit A — Problem Solving and Logic Development

- Q1.** Add two numbers (algorithm, flowchart description & program)
- Q2.** Check whether a number is even or odd
- Q3.** Find the largest of three numbers
- Q4.** Calculate simple interest
- Q5.** Check whether a number is prime
- Q6.** Find the factorial of a number
- Q7.** Swap two numbers using a temporary variable

Unit B — Introduction to C Programming

- Q8.** Basic "Hello, World!" program & C program structure
- Q9.** Demonstrate data types and their sizes using sizeof
- Q10.** Demonstrate constants using #define and const
- Q11.** Demonstrate variables and identifiers
- Q12.** Demonstrate pointer as a data type
- Q13.** Demonstrate keywords while computing area of a rectangle
- Q14.** Identify tokens in a small C program

Unit C — Operators and Expressions

- Q15.** Demonstrate arithmetic operators
- Q16.** Demonstrate relational operators
- Q17.** Demonstrate logical operators (voting eligibility)
- Q18.** Demonstrate assignment operators
- Q19.** Demonstrate unary operators
- Q20.** Demonstrate increment/decrement (pre vs post)
- Q21.** Demonstrate the conditional (ternary) operator

Unit A · Problem Solving and Logic Development

Q1. Add Two Numbers

Problem Statement

Write an algorithm, describe the flowchart, and write a C program to read two numbers from the user and print their sum.

Algorithm

1. Start
2. Read two numbers a and b
3. Compute $\text{sum} = a + b$
4. Print sum
5. Stop

Flowchart (described)

Start (oval) → **Input a, b** (parallelogram) → **Process: $\text{sum} = a + b$** (rectangle) → **Output sum** (parallelogram) → **Stop** (oval).

Solution Program

```
#include <stdio.h>

int main() {
    int a, b, sum;

    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);

    sum = a + b;
    printf("Sum = %d\n", sum);

    return 0;
}
```

Sample Output

```
Enter two numbers: 12 8
Sum = 20
```

Explanation:

- `scanf` reads two integers into `a` and `b` using their addresses (`&a`, `&b`).
- The algorithm's three logical steps (input → process → output) map directly onto three lines of C code — this is the essence of translating logic into a program.

Q2. Check Even or Odd

Problem Statement

Write an algorithm and C program to read a number and determine whether it is even or odd.

Algorithm

1. Start
2. Read number `n`
3. If `n % 2 == 0`, print "Even"; otherwise print "Odd"
4. Stop

Solution Program

```
#include <stdio.h>

int main() {
    int n;

    printf("Enter a number: ");
    scanf("%d", &n);

    if (n % 2 == 0)
        printf("%d is Even\n", n);
    else
        printf("%d is Odd\n", n);

    return 0;
}
```

Sample Output

```
Enter a number: 7
7 is Odd
```

Explanation:

- % is the modulus operator — it gives the remainder after division.
- Any number divisible by 2 leaves a remainder of 0, so $n \% 2 == 0$ is the decision condition (a "decision box" in a flowchart).

Q3. Largest of Three Numbers

Problem Statement

Write an algorithm and C program to read three numbers and print the largest among them.

Algorithm

1. Start
2. Read a, b, c
3. If $a > b$ and $a > c$, largest = a
4. Else if $b > c$, largest = b; else largest = c
5. Print largest
6. Stop

Solution Program

```
#include <stdio.h>

int main() {
    int a, b, c;

    printf("Enter three numbers: ");
    scanf("%d %d %d", &a, &b, &c);

    if (a >= b && a >= c)
        printf("Largest = %d\n", a);
    else if (b >= c)
        printf("Largest = %d\n", b);
    else
        printf("Largest = %d\n", c);

    return 0;
}
```

Sample Output

```
Enter three numbers: 4 9 6
Largest = 9
```

Explanation:

- The nested if...else if...else chain checks conditions in sequence, exactly like a flowchart's chain of decision diamonds.
- Using $\geq$ instead of $>$ correctly handles the case where two or more numbers are equal and the largest.

Q4. Simple Interest

Problem Statement

Write an algorithm and C program to calculate simple interest given principal, rate, and time, using the formula $SI = (P \times R \times T) / 100$.

Algorithm

1. Start
2. Read P, R, T
3. Compute $SI = (P * R * T) / 100$
4. Print SI
5. Stop

Solution Program

```
#include <stdio.h>

int main() {
    float p, r, t, si;

    printf("Enter principal, rate, time: ");
    scanf("%f %f %f", &p, &r, &t);
```

```

    si = (p * r * t) / 100;
    printf("Simple Interest = %.2f\n", si);

    return 0;
}

```

Sample Output

```

Enter principal, rate, time: 1000 5 2
Simple Interest = 100.00

```

Explanation:

- float is used because interest calculations often involve decimal values.
- %.2f formats the output to 2 decimal places for currency-style precision.

Q5. Check Prime Number

Problem Statement

Write an algorithm and C program to check whether a given number is prime.

Algorithm

1. Start
2. Read n
3. Set isPrime = 1; if n < 2, set isPrime = 0
4. For i = 2 to n/2: if n % i == 0, set isPrime = 0 and stop checking
5. If isPrime == 1, print "Prime"; else print "Not Prime"
6. Stop

Solution Program

```

#include <stdio.h>

int main() {
    int n, isPrime = 1;

    printf("Enter a number: ");
    scanf("%d", &n);

    if (n < 2)
        isPrime = 0;

    for (int i = 2; i <= n / 2; i++) {
        if (n % i == 0) {
            isPrime = 0;
            break;
        }
    }

    if (isPrime)
        printf("%d is Prime\n", n);
    else
        printf("%d is Not Prime\n", n);

    return 0;
}

```

Sample Output

```

Enter a number: 17
17 is Prime

```

Explanation:

- A prime number has no divisors other than 1 and itself, so the loop tries every possible divisor from 2 up to n/2.
- break exits the loop early the moment a divisor is found — there's no need to keep checking.

Q6. Factorial of a Number

Problem Statement

Write an algorithm and C program to compute the factorial of a number ($n! = n \times (n-1) \times \dots \times 1$).

Algorithm

1. Start
2. Read n
3. Set fact = 1

4. For i = 1 to n: fact = fact * i
5. Print fact
6. Stop

Solution Program

```
#include <stdio.h>

int main() {
    int n;
    long int fact = 1;

    printf("Enter a number: ");
    scanf("%d", &n);

    for (int i = 1; i <= n; i++) {
        fact = fact * i;
    }

    printf("Factorial = %ld\n", fact);

    return 0;
}
```

Sample Output

```
Enter a number: 5
Factorial = 120
```

Explanation:

- fact is declared as long int because factorial values grow very quickly and can overflow a regular int.
- The loop multiplies fact by each number from 1 to n, accumulating the product — a classic "running total" pattern.

Q7. Swap Two Numbers

Problem Statement

Write an algorithm and C program to swap the values of two variables using a temporary (third) variable, and identify this as an example of structured programming.

Algorithm

1. Start
2. Read a, b
3. temp = a
4. a = b
5. b = temp
6. Print a, b
7. Stop

Solution Program

```
#include <stdio.h>

int main() {
    int a, b, temp;

    printf("Enter a and b: ");
    scanf("%d %d", &a, &b);

    temp = a;
    a = b;
    b = temp;

    printf("After swap: a = %d, b = %d\n", a, b);

    return 0;
}
```

Sample Output

```
Enter a and b: 3 8
After swap: a = 8, b = 3
```

Explanation:

- Without a temporary variable, assigning a = b would overwrite a's original value before it's copied into b — temp preserves it.
- This program follows **structured programming** principles: it runs top-to-bottom in a single sequence with no jumps (goto), making it easy to read and verify.

Unit B · Introduction to C Programming

Q8. Hello World & Program Structure

Problem Statement

Write a basic C program that prints "Hello, World!" and label the standard sections of a C program's structure.

Solution Program

```
// 1. Preprocessor directive
#include <stdio.h>

// 2. main() function — entry point of every C program
int main() {
    // 3. Statements inside the function body
    printf("Hello, World!\n");

    // 4. Return statement
    return 0;
}
```

Sample Output

```
Hello, World!
```

Explanation:

- `#include <stdio.h>` is a preprocessor directive that brings in declarations for input/output functions like `printf`.
- `int main()` is the mandatory entry point — execution always starts here.
- `return 0;` tells the operating system the program finished successfully.

Q9. Data Types and sizeof

Problem Statement

Write a C program that declares variables of type `int`, `float`, `char`, and `double`, and prints the memory size occupied by each using the `sizeof` operator.

Solution Program

```
#include <stdio.h>

int main() {
    int a = 10;
    float b = 5.5;
    char c = 'A';
    double d = 9.99;

    printf("Size of int: %lu bytes\n", sizeof(a));
    printf("Size of float: %lu bytes\n", sizeof(b));
    printf("Size of char: %lu bytes\n", sizeof(c));
    printf("Size of double: %lu bytes\n", sizeof(d));

    return 0;
}
```

Sample Output

```
Size of int: 4 bytes
Size of float: 4 bytes
Size of char: 1 bytes
Size of double: 8 bytes
```

Explanation:

- Each data type reserves a different amount of memory, which limits the range of values it can hold.
- `sizeof` is a compile-time operator (not a function call) that returns the size in bytes of its operand's type.

Q10. Constants — `#define` and `const`

Problem Statement

Write a C program to calculate the area of a circle using a constant value of `PI`, demonstrating both `#define` and the `const` keyword.

Solution Program

```
#include <stdio.h>
#define PI 3.14159

int main() {
    const int radius = 5;
    float area;

    area = PI * radius * radius;
    printf("Area of circle = %.2f\n", area);

    return 0;
}
```

Sample Output

```
Area of circle = 78.54
```

Explanation:

- `#define PI 3.14159` is a preprocessor macro — before compilation, every occurrence of `PI` in the code is textually replaced with `3.14159`.
- `const int radius = 5;` creates a real variable that the compiler prevents from being modified anywhere later in the program.

Q11. Variables and Identifiers

Problem Statement

Write a C program that declares variables with valid identifier names, and in comments show examples of invalid identifier names with reasons.

Solution Program

```
#include <stdio.h>

int main() {
    int studentAge = 20;        // valid: starts with a letter
    float _percentage = 88.5;   // valid: starts with underscore
    int total_marks = 450;      // valid: letters, digits, underscore

    // Invalid examples (would cause compiler errors if uncommented):
    // int 2ndYear;           -> cannot start with a digit
    // float total-fee;       -> hyphen not allowed
    // int int;               -> cannot use a C keyword as an identifier

    printf("Age: %d, Percentage: %.1f, Marks: %d\n",
           studentAge, _percentage, total_marks);

    return 0;
}
```

Sample Output

```
Age: 20, Percentage: 88.5, Marks: 450
```

Explanation:

- An **identifier** is the name given to a variable, function, or other entity. It must start with a letter or underscore and may contain letters, digits, and underscores.
- Identifiers cannot be a reserved **keyword** (like `int`, `return`, `for`) since these have special meaning to the compiler.

Q12. Pointer as a Data Type

Problem Statement

Write a C program to declare an integer variable and a pointer to it, demonstrating that a pointer is itself a distinct data type used to store an address.

Solution Program

```
#include <stdio.h>

int main() {
    int num = 42;
    int *ptr = &num;    // ptr is of type "pointer to int"

    printf("Value of num: %d\n", num);
    printf("Address of num: %p\n", &num);
    printf("Value stored in ptr: %p\n", ptr);
    printf("Size of ptr (bytes): %lu\n", sizeof(ptr));

    return 0;
}
```

Sample Output

```
Value of num: 42
Address of num: 0x1000
Value stored in ptr: 0x1000
Size of ptr (bytes): 8
```

Explanation:

- `int *ptr` is not an `int` — it is its own data type, "pointer to `int`," which stores an address rather than an ordinary value.
- On most modern 64-bit systems, every pointer (regardless of what it points to) occupies 8 bytes, since it just stores an address.

Q13. Keywords — Area of a Rectangle

Problem Statement

Write a C program to compute the area of a rectangle, and identify each reserved keyword used in the program.

Solution Program

```
#include <stdio.h>

int main() {
    int length = 12, width = 5, area;

    area = length * width;
    printf("Area = %d\n", area);

    return 0;
}
```

Sample Output

```
Area = 60
```

Explanation:

- **Keywords** used here: `int` (a data type keyword) and `return` (a control keyword) — both are reserved and cannot be renamed or reused as identifiers.
- Everything else (`length`, `width`, `area`, `main`) is a programmer-chosen identifier, not a keyword.

Q14. Identifying Tokens

Problem Statement

Consider a small C program and identify its tokens: keywords, identifiers, constants, operators, and special symbols.

Program to Analyze

```
int total = 10 + 5;
```

Token Breakdown

Token	Category
<code>int</code>	Keyword
<code>total</code>	Identifier
<code>=</code>	Operator (assignment)
<code>10, 5</code>	Constants
<code>+</code>	Operator (arithmetic)
<code>;</code>	Special symbol (statement terminator)

Explanation:

- A **token** is the smallest individual unit in a C program that the compiler recognizes — every program is broken down into a stream of tokens before compilation continues.
- C recognizes six token categories: keywords, identifiers, constants, string literals, operators, and special symbols.

Unit C · Operators and Expressions

Q15. Arithmetic Operators

Problem Statement

Write a C program to demonstrate all the arithmetic operators (+, -, *, /, %) on two integers.

Solution Program

```
#include <stdio.h>

int main() {
    int a = 17, b = 5;

    printf("Sum = %d\n", a + b);
    printf("Difference = %d\n", a - b);
    printf("Product = %d\n", a * b);
    printf("Quotient = %d\n", a / b);
    printf("Remainder = %d\n", a % b);

    return 0;
}
```

Sample Output

```
Sum = 22
Difference = 12
Product = 85
Quotient = 3
Remainder = 2
```

Explanation:

- Since both a and b are integers, a / b performs **integer division**, discarding the decimal part ($17/5 = 3$, not 3.4).
- % gives the remainder of that division and only works with integer operands.

Q16. Relational Operators

Problem Statement

Write a C program to compare two numbers using all relational operators (>, <, >=, <=, ==, !=).

Solution Program

```
#include <stdio.h>

int main() {
    int a = 10, b = 20;

    printf("a > b : %d\n", a > b);
    printf("a < b : %d\n", a < b);
    printf("a >= b : %d\n", a >= b);
    printf("a <= b : %d\n", a <= b);
    printf("a == b : %d\n", a == b);
    printf("a != b : %d\n", a != b);

    return 0;
}
```

Sample Output

```
a > b : 0
a < b : 1
a >= b : 0
a <= b : 1
a == b : 0
a != b : 1
```

Explanation:

- Relational operators compare two values and produce a Boolean-style result: 1 for true, 0 for false (C has no separate boolean type in its classic form).
- These are the building blocks of every if condition and loop test.

Q17. Logical Operators — Voting Eligibility

Problem Statement

Write a C program that checks a person's voting eligibility: a person is eligible only if their age is 18 or above **and** they are a citizen. Use logical operators.

Solution Program

```
#include <stdio.h>

int main() {
    int age = 20, isCitizen = 1;    // 1 = yes, 0 = no

    if (age >= 18 && isCitizen)
        printf("Eligible to vote.\n");
    else
        printf("Not eligible to vote.\n");

    if (!isCitizen)
        printf("Reason: not a citizen.\n");

    return 0;
}
```

Sample Output

```
Eligible to vote.
```

Explanation:

- && (logical AND) requires **both** conditions to be true for the overall expression to be true.
- ! (logical NOT) flips a condition's truth value — !isCitizen is true only when isCitizen is 0.
- || (logical OR, not shown above) would require only **one** of the conditions to be true.

Q18. Assignment Operators

Problem Statement

Write a C program to demonstrate the compound assignment operators (+=, -=, *=, /=, %=).

Solution Program

```
#include <stdio.h>

int main() {
    int a = 10;

    a += 5; printf("After += 5:  a = %d\n", a);
    a -= 3; printf("After -= 3:  a = %d\n", a);
    a *= 2; printf("After *= 2:  a = %d\n", a);
    a /= 4; printf("After /= 4:  a = %d\n", a);
    a %= 3; printf("After %= 3:  a = %d\n", a);

    return 0;
}
```

Sample Output

```
After += 5:  a = 15
After -= 3:  a = 12
After *= 2:  a = 24
After /= 4:  a = 6
After %= 3:  a = 0
```

Explanation:

- a += 5; is shorthand for a = a + 5; — every compound assignment operator follows this same "do the operation, then store it back" pattern.
- These operators make code shorter and are especially common in loop counters and accumulators.

Q19. Unary Operators

Problem Statement

Write a C program to demonstrate unary operators: unary minus, logical NOT, and sizeof.

Solution Program

```
#include <stdio.h>

int main() {
    int a = 7;

    printf("Unary minus: %d\n", -a);
    printf("Logical NOT of a: %d\n", !a);
}
```

```

    printf("Logical NOT of 0: %d\n", !0);
    printf("Size of a: %lu bytes\n", sizeof(a));

    return 0;
}

```

Sample Output

```

Unary minus: -7
Logical NOT of a: 0
Logical NOT of 0: 1
Size of a: 4 bytes

```

Explanation:

- A **unary operator** acts on a single operand — unlike + or &&, which need two.
- Unary minus (-a) negates a value; ! inverts a truth value (non-zero becomes 0, and 0 becomes 1); sizeof returns a type's size in bytes.

Q20. Increment/Decrement — Pre vs Post

Problem Statement

Write a C program to show the difference between pre-increment (++a) and post-increment (a++), and similarly for decrement.

Solution Program

```

#include <stdio.h>

int main() {
    int a = 5, b = 5;

    printf("Post-increment: a++ = %d, then a = %d\n", a++, a);
    printf("Pre-increment:  ++b = %d, then b = %d\n", ++b, b);

    return 0;
}

```

Sample Output

```

Post-increment: a++ = 5, then a = 6
Pre-increment:  ++b = 6, then b = 6

```

Explanation:

- a++ (post-increment) uses the **current** value of a in the expression first, and increments it afterward.
- ++b (pre-increment) increments b **first**, then uses the new value in the expression.
- Both eventually increase the variable by 1 — the difference only matters when the operator is used inside a larger expression.

Q21. Conditional (Ternary) Operator

Problem Statement

Write a C program to find the maximum of two numbers using the conditional (ternary) operator ? : instead of an if-else statement.

Solution Program

```

#include <stdio.h>

int main() {
    int a = 15, b = 9, max;

    max = (a > b) ? a : b;
    printf("Maximum = %d\n", max);

    return 0;
}

```

Sample Output

```

Maximum = 15

```

Explanation:

- The ternary operator has the form condition ? value_if_true : value_if_false — it's a compact expression version of a simple if-else.
- Here, if a > b is true, max gets a; otherwise it gets b — all in a single line.